

ASSESSMENT 3

Theme: User Login Automation System

Section A: Conceptual Questions

1. You are automating a login feature using Java and Selenium. Explain why Java is suitable for building reusable automation logic.

ANS.

Java is suitable for Selenium automation because it is an object-oriented programming language that supports code reusability through classes, objects, inheritance, and methods. Common functions such as login, browser setup, and utility methods can be written once and reused across multiple test cases. Java also has a large collection of libraries, strong exception handling, and excellent integration with Selenium, TestNG/JUnit, and Maven, making automation scripts efficient, maintainable, and scalable.

2. During automation execution, login validation fails intermittently. Explain how control statements can be used to handle different outcomes.

Ans.

Control statements help the automation script make decisions based on different conditions.

For example:

if-else statements check whether the login is successful or failed.

switch statements handle different application responses or user roles.

Loops (for, while) can retry login attempts or wait until an element becomes available.

These control statements allow the script to respond appropriately instead of stopping when different outcomes occur.

3. Error messages on the login page change dynamically. Explain how string handling helps validate correct messages.

Ans.

String handling allows automation scripts to compare, search, and manipulate text displayed on the web page. Java methods such as `equals()`, `contains()`, `startsWith()`, `endsWith()`, and `trim()` help verify whether the displayed error message matches the expected result, even when only part of the message changes dynamically. This makes validation more flexible and reliable.

4. Explain how object-oriented concepts improve maintainability in automation frameworks.

Ans.

Object-oriented programming (OOP) improves maintainability by organizing automation code into reusable classes and objects. Key concepts include:

Encapsulation: Keeps data and methods together, improving security and organization.

Inheritance: Allows common functionality to be reused in child classes, reducing duplicate code.

Polymorphism: Enables the same method to perform different tasks based on the object.

Abstraction: Hides implementation details and exposes only necessary functionality.

These concepts make automation frameworks easier to update, extend, and maintain.

5. An automation script fails due to invalid input. Explain how exception handling helps prevent script termination.

Ans.

Exception handling in Java uses `try`, `catch`, `finally`, `throw`, and `throws` to manage runtime errors. When invalid input or unexpected events occur, the exception is caught and handled without terminating the entire automation script. The script can log the error, take a screenshot, clean up resources, and continue executing the remaining test cases, making the automation framework more stable and reliable.

Section B: Practical Tasks

1. Conditional Login Validation: Write a Java program using Scanner to accept credentials. Use if-else blocks to compare inputs against a hardcoded valid username and password, printing "Login Successful" or "Access Denied."

Ans.

JAVA PROGRAM :-

```
import java.util.Scanner;

public class LoginValidation {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String validUsername = "admin";
        String validPassword = "admin123";

        System.out.print("Enter Username: ");
        String username = sc.nextLine();

        System.out.print("Enter Password: ");
        String password = sc.nextLine();

        if (username.equals(validUsername) &&
password.equals(validPassword)) {
            System.out.println("Login Successful");
        }
    }
}
```

```

    } else {
        System.out.println("Access Denied");
    }

    sc.close();
}
}

```

2. Multi-User Testing (Arrays/Collections): Store multiple sets of credentials in a HashMap or a 2D Array. Use a for-each loop to iterate through the collection and attempt a login for each pair, simulating a batch test.

ANS.

JAVA PROGRAM :-

```

import java.util.HashMap;
import java.util.Map;

public class MultiUserLogin {

    public static void main(String[] args) {

        HashMap<String, String> users = new HashMap<>();

        users.put("admin", "admin123");
        users.put("john", "john123");
        users.put("alice", "alice123");
    }
}

```

```

for (Map.Entry<String, String> user : users.entrySet()) {

    String username = user.getKey();
    String password = user.getValue();

    System.out.println("Testing Login:");
    System.out.println("Username: " + username);
    System.out.println("Password: " + password);
    System.out.println("Login Successful");
    System.out.println("-----");
}
}
}

```

3. Method Overloading: Demonstrate overloading by creating multiple login() methods. For example:

- login(String username, String password)
- login(String username, String password, int otp)
- login(long phoneNumber, String password)

ANS.

Java program :-

```

public class LoginSystem {

    // Login using username and password
    void login(String username, String password) {
        System.out.println("Login using Username and Password");
        System.out.println("Username: " + username);
    }
}

```

```

    }

    // Login using username, password, and OTP
    void login(String username, String password, int otp) {
        System.out.println("Login using Username, Password, and OTP");
        System.out.println("Username: " + username);
        System.out.println("OTP: " + otp);
    }

    // Login using phone number and password
    void login(long phoneNumber, String password) {
        System.out.println("Login using Phone Number and Password");
        System.out.println("Phone Number: " + phoneNumber);
    }

    public static void main(String[] args) {

        LoginSystem obj = new LoginSystem();

        obj.login("admin", "admin123");

        obj.login("admin", "admin123", 123456);

        obj.login(9876543210L, "mobile123");
    }
}

```

4. Exception Handling: Implement a custom exception (e.g., InvalidLoginException). Use a try-catch block to wrap the validation logic so that if a login fails, the program throws the exception and handles it without crashing.

Ans.

Java program :-

```
import java.util.Scanner;
```

```
// Custom Exception
```

```
class InvalidLoginException extends Exception {
```

```
    public InvalidLoginException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

```
public class LoginExceptionDemo {
```

```
    public static void validateLogin(String username, String password)
```

```
        throws InvalidLoginException {
```

```
        String validUsername = "admin";
```

```
        String validPassword = "admin123";
```

```
        if (username.equals(validUsername) &&  
password.equals(validPassword)) {
```

```
            System.out.println("Login Successful");
```

```
        } else {
```

```
            throw new InvalidLoginException("Invalid Username or Password");
```

```

    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter Username: ");
    String username = sc.nextLine();

    System.out.print("Enter Password: ");
    String password = sc.nextLine();

    try {
        validateLogin(username, password);
    }
    catch (InvalidLoginException e) {
        System.out.println("Login Failed");
        System.out.println("Exception: " + e.getMessage());
    }
    finally {
        System.out.println("Program Executed Successfully.");
        sc.close();
    }
}
}

```


5. File I/O Logging: Write a Java program using FileWriter or BufferedWriter to record the outcome of each login attempt. The program must generate a results.txt file containing timestamps and the "Pass/Fail" status of the execution.

Ans.

Java program :-

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;

public class LoginLogger {

    public static void main(String[] args) {

        String username = "admin";
        String password = "admin123";

        String validUsername = "admin";
        String validPassword = "admin123";

        String result;

        if (username.equals(validUsername) &&
password.equals(validPassword)) {
            result = "Pass";
        } else {
            result = "Fail";
        }
    }
}
```

```
try {

    BufferedWriter writer = new BufferedWriter(new
    FileWriter("results.txt", true));

    writer.write("Timestamp : " + LocalDateTime.now());
    writer.newLine();

    writer.write("Username : " + username);
    writer.newLine();

    writer.write("Result   : " + result);
    writer.newLine();

    writer.write("-----");
    writer.newLine();

    writer.close();

    System.out.println("Login result recorded in results.txt");

} catch (IOException e) {

    System.out.println("Error writing to file: " + e.getMessage());

}
}
```

Section C: Mini Capstone Project – End-to-End Automation

Simulation Context You are assigned as an Automation Test Engineer to simulate an end-to-end automation flow using Core Java concepts.

1. Test Planning Document: Prepare a Test Plan covering automation scope, tools used, test data strategy, risks, and execution approach.

ANS.

Test Planning Document :-

Test Plan

Project Name :- User Login Automation System

Objective :-

To automate the login functionality of a web application using Java and Selenium and verify that users can log in, handle invalid credentials, and log out successfully.

Scope :-

In Scope

- Application launch
- Login with valid credentials
- Login with invalid credentials
- Error message validation
- Logout functionality

Out of Scope

- User registration
- Password reset
- Performance and security testing

Tools Used :-

- Programming Language: Java

- Automation Tool: Selenium WebDriver
- IDE: Eclipse / IntelliJ IDEA
- Testing Framework: TestNG or JUnit
- Build Tool: Maven
- Browser: Google Chrome
- Version Control: Git

Test Data Strategy :-

Test Case	Username	Password	Expected Result
Valid Login.	admin	admin123	Login Successful
Invalid Password	admin	wrong123	Error Message
Invalid Username	user123	admin123	Error Message
Empty Fields	Blank	Blank	Validation Message

Risks :-

- Dynamic web elements
- Browser compatibility issues
- Network interruptions
- Incorrect test data

Execution Approach :-

1. Launch browser.
2. Open application URL.
3. Enter test data.
4. Validate login result.
5. Handle errors if login fails.
6. Perform logout.
7. Close browser.
8. Generate execution report.

2. Requirements Traceability Matrix: Create a Requirements Traceability Matrix mapping login requirements to Java methods and test scenarios.

Ans.

Requirements Traceability Matrix :-

R.ID	Requirement	Java Method	Test Scenario	Status
R1	Launch Application	launchBrowser()	Verify application opens successfully	Pass
R2	Enter Username	enterUsername()	Enter valid/invalid username	Pass
R3	Enter Password	enterPassword()	Enter valid/invalid password	Pass
R4	Login Validation	clickLogin()	Verify successful login	Pass
R5	Error Validation	ErrorMessage()	Verify invalid login message	Pass
R6	Logout	logout()	Verify successful logout	Pass

3. End-to-End Test Scenario: Write one End-to-End test scenario covering: Application Launch → Login Validation → Error Handling → Logout.

Ans.

Test Scenario :- User Login and Logout

Test Case ID :- TC_Login_001

Title :- Verify complete Login and Logout functionality.

Preconditions :-

- Application is available.
- Valid user credentials are available.

Test Steps :-

Step	Action	Expected Result
1	Launch browser	Browser opens successfully
2	Open application URL	Login page is displayed
3	Enter username	Username accepted
4	Enter password	Password accepted

- | | | |
|---|---------------|------------------------------|
| 5 | Click Login | Dashboard displayed |
| 6 | Click Logout | User logged out successfully |
| 7 | Close browser | Browser closes |

Expected Result :-

- Valid credentials should log in successfully.
- Invalid credentials should display an error message.
- Exceptions should be handled without terminating the script.
- Logout should return the user to the login page.

4. Test Summary Report: Prepare a Test Summary Report including execution status, failures handled, and readiness for automation execution.

ANS.

Test Summary Report :-

Project

User Login Automation System

Execution Summary

Total Test Cases Executed	Passed	Failed	Blocked
4	4	0	0

Failures Handled

- Invalid username
- Invalid password
- Empty input fields
- Element not found exception using exception handling

Automation Readiness

- ✓ Test scripts created using Java and Selenium.
- ✓ Reusable methods implemented.
- ✓ Exception handling included.
- ✓ Login and logout functionality validated.
- ✓ Framework ready for execution and future enhancements.